2. g(n): Depth of the node, h(n): Manhattan Distanc

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

# Manhattan Distance heuristic: h(n)
def h(state):
    dist = 0
    for i, tile in enumerate(state):
        if tile != 0:
            target = tile - 1
            dist += abs(i // 3 - target // 3) + abs(i % 3 - target % 3)
    return dist

def get_neighbors(state):
    i = state.index(0)
    r, c = i // 3, i % 3
    moves = [(-1, 0, 'UP'), (1, 0, 'DOWN'), (0, -1, 'LEFT'), (0, 1, 'RIGHT')]
    neighbors = []
    for dr, dc, action in moves:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            ni = nr * 3 + nc
            new_state = list(state)
            new_state[i], new_state[ni] = new_state[ni], new_state[i]
            neighbors.append((tuple(new_state), action))
    return neighbors
```

```
def a_star(start):
    # Store: (f, g, state, path_history)
    # Each step in history: (state, action_taken, g, h, f)
    queue = [(h(start), 0, start, [(start, 'START', 0, h(start), h(start))])]
    visited = set()
    expanded_nodes = 0

    while queue:
        f, g, current, history = heapq.heappop(queue)

        if current == GOAL:
            return history, expanded_nodes

        if current in visited:
            continue
        visited.add(current)
        expanded_nodes += 1

        for next_state, action in get_neighbors(current):
            if next_state not in visited:
                next_g = g + 1
                next_h = h(next_state)
                next_f = next_g + next_h
                next_history = history + [(next_state, action, next_g, next_h, next_f)]
                heapq.heappush(queue, (next_f, next_g, next_state, next_history))

def print_board(state):
```

```
    for i in range(0, 9, 3):
        row = [str(x) if x != 0 else ' ' for x in state[i:i+3]]
        print(f"  +---+---+---+\n  | {' | '.join(row)} |")
    print("  +---+---+---+")

# --- Execution ---
start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
history, total_expanded = a_star(start)

print("=" * 45)
print("      8-PUZZLE SOLVER (A* MANHATTAN)     ")
print("=" * 45)

for step_num, (state, action, g, h_val, f_val) in enumerate(history):
    print(f"\nStep {step_num}: Action = {action}")
    print(f"g(n) [Depth] = {g} | h(n) [Manhattan] = {h_val} | f(n) = {f_val}")
    print_board(state)

print("=" * 45)
print(f"Search Completed Successfully!")
print(f"Total Steps to Goal : {len(history) - 1}")
print(f"Total Nodes Expanded: {total_expanded}")
print("=" * 45)
```

```
Code
```

```
==============================================
       8-PUZZLE SOLVER (A* MANHATTAN)
==============================================

Step 0: Action = START
g(n) [Depth] = 0 | h(n) [Manhattan] = 2 | f(n) = 2
  +---+---+---+
  | 1 | 2 | 3 |
  +---+---+---+
  | 4 |   | 6 |
  +---+---+---+
  | 7 | 5 | 8 |
  +---+---+---+

Step 1: Action = DOWN
g(n) [Depth] = 1 | h(n) [Manhattan] = 1 | f(n) = 2
  +---+---+---+
  | 1 | 2 | 3 |
  +---+---+---+
  | 4 | 5 | 6 |
  +---+---+---+
  | 7 |   | 8 |
  +---+---+---+

Step 2: Action = RIGHT
g(n) [Depth] = 2 | h(n) [Manhattan] = 0 | f(n) = 2
  +---+---+---+
  | 1 | 2 | 3 |
  +---+---+---+
  | 4 | 5 | 6 |
  +---+---+---+
  | 7 | 8 |   |
  +---+---+---+
==============================================
Search Completed Successfully!
Total Steps to Goal : 2
Total Nodes Expanded: 2
==============================================
```

```
[ ]:
```